

TABLE-2

Generating design of other traditional database model
 creating hierarchical network model of the database by
 enhancing the sound abstract data by performing following
 tasks using forms of inheritance:

- 2.a. Identify the specificity of each relationship, find and form surplus relations.
- 2.b. Check is-a hierarchy / has-a hierarchy and perform generalization and/or specialization relationship.
- 2.c. Find the domain of the attribute and perform a check constraint to the applicable.
- 2.d. Rename the relations.
- 2.e. Perform SQL relations using DDL, DCL commands.

IMPLEMENTATION OF DDL, DML, DCL and TCL COMMANDS OF SQL

Aim:- Implementation of DDL, DML, DCL and TCL commands of SQL with suitable examples

Objective:

- * To understand the different issues involved in the design and implementation of a database system.
- * To understand and use data definition language to write query for a database

Theory:

Oracle has many tools such as SQL*PLUS, Oracle Forms, Oracle Report writer, Oracle Graphics etc.

SQL*PLUS: The SQL*PLUS tool is made up of two distinct parts.

These are

Interactive SQL: Interactive SQL is designed for create, access and manipulate data structures like tables and indexes.

PL/SQL: PL/SQL can be used to develop programs for different applications.

Oracle Forms: This tool allows you to create a data entry screen along with the suitable menu objects. Thus it is the Oracle forms tool that handles data gathering and data validation in a commercial application.

Report Writer: Report writer allows programmers to prepare innovative reports using data from the Oracle structure like tables, views etc. It is the report writer tool that handles the reporting section of commercial application.

Oracle Graphics: Some of the data can be better represented in the form of pictures. The Oracle graphics tool allows programmers to prepare graphs using data from Oracle structures like tables, views etc.

SQL (Structured Query Language):

Structured Query Language is a database computer language designed for managing data in relational database management systems (RDBMS), and originally based upon Relational Algebra. Its scope includes data query and update, schema creation and modification, and data access control.

SQL was one of the first languages for Edgar F. Codd's relational model and became the most widely used language for relational database.

IBM developed SQL in mid of 1970's

Oracle incorporated in the year 1979.

SQL used by IBM/DB2 and as database systems.

SQL adopted as standard language for RDBS by ANSI in 1989.

DATA TYPES:

1. CHAR (size): This data type is used to store character strings values of fixed length. The size in brackets determines the number of characters the cell can hold. The maximum number of character is 255 characters.

2. VARCHAR (size) / VARCHAR2 (size): This data type is used to store number (fixed or floating point). Number of virtually any magnitude may be stored up to 38 digits of precision.

3. NUMBER (p,s): The NUMBER data type is used to store number (fixed or floating point). Number of virtually any magnitude may be stored up to 38 digits of precision.

Number as large as 9.99×10^{124} . The precision (p) determine the number of places to the right of the decimal. If scale is omitted then the default is zero. If precision is omitted, values are stored with their original precision up to the maximum of 38 digits.

4. DATE: This data types is used to represent date and time. The standard format is DD-MM-YY as in 17-SEP-2009. To enter dates other than the standard format, use the

appropriate functions. Date time stores date in the 24-hour format. By default the time in a date field is 12:00:00 am, if no time portion is specified. The default date for a date field is the first day the current month.

5. LONG: This data type is used to store variable length character strings containing up to 2GB. Long data can be used to store arrays of binary data in ASCII format. Long values cannot be indexed, and the normal character functions such as SUBSTR cannot be applied.

6. RAW: The RAW data type is used to store binary data, such as digitized picture or image. Data loaded into columns of these data types are stored without any further conversion. RAW data types can have a minimum length of 255 bytes. LONG RAW data type can contain up to 2GB.

DATA DEFINITION LANGUAGES: (for Shopping Management System)

A DATA Definition Language (DDL) statement are used to define the database structure or schema.

CREATE: To make a new database, table, index, or stored query. A create statement in SQL creates an object inside of a relational database management system (RDBMS).

SYNTAX:

CREATE TABLE table_name

column_name1 data_type (size), column_name2 data_type (size) ...
column_name-n data_type (size);

ALTER: - To modify an existing database object. alter the structure of the database.

- To add a column in a table

SYNTAX

ALTER TABLE table-name ADD column-name datatype([size]);

- To delete a column in a table

SYNTAX

ALTER TABLE table-name DROP column-name;

- To modify a column in a table

SYNTAX:

ALTER TABLE table-name MODIFY column-name data-type ([new-size]);

RENAME

SYNTAX

ALTER TABLE table-name RENAME column old-column-name to new-column-name;

DESCRIBE

SYNTAX

DESC table-name

TRUNCATE TABLE:

Remove all records from a table, including all spaces allocated for the records are removed

SYNTAX

TRUNCATE TABLE table-name

DATA MANIPULATION LANGUAGES:

Data manipulation language allows the users to query and manipulate data in existing schema in object. It allows following data to insert, delete, update and recovery data in schema object.

INSERT:- Values can be inserted into table using insert commands. There are two types of insert commands. They are multiple value insert commands (using &' symbol) single value insert command (without using &' symbol).

SYNTAX

INSERT INTO table_name VALUES (value1, value2, value3, ...);
(OR)

INSERT INTO table_name (column1, column2, column3, ...) VALUES
value1, value2, value3, ...);

UPDATE:- This allows the user to update the particular column value using the where clause condition.

SYNTAX

UPDATE <table-name> SET <COL1 = value> WHERE <column=value>;

DELETE:- This allows you to delete the particular column values using where clause condition.

SYNTAX:

DELETE FROM <table-name> WHERE <conditions>;

SELECT:- The select statement is used to query a database. This statement is used to retrieve the information from the database. The SELECT statement can be used in many ways. They are

1. Selecting some columns:

To select specified number of columns from the table the following command is used

SYNTAX

SELECT column-name FROM table-name;

2. Select using DISTINCT:-

The DISTINCT keyword is used to return only different values (i.e.) this command does not select the duplicate values from the table.

SYNTAX

SELECT DISTINCT column name(s) FROM table-name;

3. Select using BETWEEN

BETWEEN can be used to get those items that fall within a range.

SYNTAX

SELECT column name FROM table-name WHERE column name BETWEEN Value 1 AND Value 2;

4. Renaming:

The select statement can be used to rename either a column or the entire table.

SYNTAX

Renaming a column:

SELECT column name AS new name FROM table-name;

5. SORTING

The select statement with the order by clause is used to sort the contents

Table either in ascending or descending order.

SYNTAX

SELECT column name FROM table_name WHERE
condition ORDER BY column name ASC/DESC;

6. To select by matching some patterns:

The select statement along with like clause is used to match strings. The like condition is used to specify a search pattern in a column.

SYNTAX:-

SELECT column name FROM table_name WHERE column
name LIKE "% or -";

% : Matches any sub string

- : Matches a single character.

7. Select using AND, OR, NOT:

We can combine one or more conditions in a SELECT statement using the logical operators AND, OR, NOT.

SYNTAX

SELECT column name FROM table_name WHERE condition 1
LOGICAL OPERATOR condition 2

DATA CONTROL LANGUAGES:

1. Create:

• create user kamal identified by kamal;
User created.

2. Grant:

• grant all privileges to kamal;
Grant succeeded.

3. Revoke:

• revoke all privileges from kamal;
Revoke succeeded.

TRANSACTION CONTROL LANGUAGES

1. commit:

- commit;

commit complete.

2. save point:

- savepoint k1;

Savepoint created

3. Rollback:

- rollback to k1;

Rollback complete

OUTPUT CREATE TABLE table-name
(column-name 1 data-type (Csize)), column-name 2 data-type
(Csize) ... column-name-n
data-type (Csize);

SQL > desc icol1

Name	NULL?	TYPE
NAME		VARCHAR(12)
UTUNO		NUMBER(5)
ADDRESS		VARCHAR(14)

SQL > desc icol1

Name	NULL?	TYPE
NAME		
UTUNO		
ADDRESS		

ALTER TABLE table-name ADD column-name datatype (Csize);

SQL > desc icol1 (to add column)

Name	NULL?	TYPE
NAME		VARCHAR(12)
UTUNO		NUMBER(5)
ADDRESS		VARCHAR(14)

ALTER TABLE table-name DROP
column (column-name);

SQL > desc icol1 (to drop column)

Name	NULL?	TYPE
NAME		VARCHAR(12)
UTUNO		NUMBER(5)
PHONE NO		NUMBER(10)

ALTER TABLE table-name MODIFY column-name data-type
 SQL> desc lcoh (to modify column) (new-size);

Name	NULL?	TYPE
NAME		VARCHAR(12)
VTUNID		NUMBER(3)
PHONE NO		NUMBER(10)

ALTER TABLE table-name RENAME COLUMN old-column-name data-type to
 SQL> desc lcoh (to rename column) new-column-name (new-size);

Name	NULL?	TYPE
STUDENT_NAME		VARCHAR(12)
VTUNID		NUMBER(3)
PHONE NO		NUMBER(10)

INSERT INTO table-name VALUES (value1, value2, ...);
 SQL> desc lcoh (to insert values)

Name	VTUNID	PHONE NO
STUDENT_NAME		
Shivapriya	289	9515221655
mahalakshmi	275	9876543210
Bhavyasree	289	9768543210

SQL> UPDATE lcoh SET student_name = 'Shivapriya' WHERE VTUNID = 289

STUDENT_NAME	VTUNID	PHONE NO
Shivapriya	289	9515221655
mahalakshmi	275	9876543210
Bhavyasree	289	9768543210

SQL> DELETE FROM lcoh WHERE STUDENT_NAME = Bhavyasree

STUDENT_NAME	VTUNID	PHONE NO
Shivapriya	289	9515221655
mahalakshmi	275	9876543210

RESULT:- Thus, The implementation of DDL,

DML, DCL and TCL Commands of

SQL with suitable examples

VEL TECH	
NO.	2
PERFORMANCE (5)	
RESULT AND ANALYSIS (5)	
VIVA VOCE (5)	
RECORD (5)	
TOTAL (20)	
SIGN WITH DATE	

SQL> SELECT VTUNID FROM KOTI;

VTUNID

289

275

SQL> SELECT DISTINCT ~~column~~ PHONEID FROM KOTI;

PHONEID

9515221655

9876543210

SQL>

SELECT * FROM KOTI WHERE VTUNID BETWEEN 280 and 290;

VTUNID

289

~~275~~

SQL> SELECT STUDENT_NAME AS People_Name FROM KOTI;

People_Name

VTUNID

PHONEID

Shivapriya

289

9515221655

Manalateshmi

275

9876543210

SQL> Create user koki identified by koki;

User created

SQL> grant all privileges to koki;

Grant Succeeded

SQL> Revoke; revoke all privileges from koki;

Revoke Succeeded

SQL> Commit;

Commit completed

SQL> Savepoint S1;

Savepoint created

SQL> Rollback to S1;

Roll back complete.

VEL TECH	
EX NO.	2
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	
TOTAL (20)	
SIGN WITH DATE	

RESULT:- Thus, the implementation of DDL, DML and TCL commands is SQL with suitable examples

SQL> connect

Enter user-name: system

Enter password:

Connected.

SQL> create table koteswarao(friends_name varchar(15),phoneno number(10),address
varchar(20),salary number(5))

2 ;

Table created.

SQL> desc koteswarao

Name	Null?	Type

FRIENDS_NAME		VARCHAR2(15)
PHONENO		NUMBER(10)
ADDRESS		VARCHAR2(20)
SALARY		NUMBER(5)

SQL> alter table koteswarao add email varchar(15);

Table altered.

SQL> desc koteswarao

Name	Null?	Type

FRIENDS_NAME		VARCHAR2(15)

PHONENO	NUMBER(10)
ADDRESS	VARCHAR2(20)
SALARY	NUMBER(5)
EMAIL	VARCHAR2(15)

SQL> alter table koteswarao drop column email;

Table altered.

SQL> desc koteswarao

Name	Null?	Type

FRIENDS_NAME		VARCHAR2(15)
PHONENO		NUMBER(10)
ADDRESS		VARCHAR2(20)
SALARY		NUMBER(5)

SQL> alter table koteswarao modify salary number(6);

Table altered.

SQL> desc koteswarao

Name	Null?	Type

FRIENDS_NAME		VARCHAR2(15)
PHONENO		NUMBER(10)

ADDRESS VARCHAR2(20)

SALARY NUMBER(6)

SQL> alter table koteswarao rename column friends_name to best_friends_name;

Table altered.

SQL> desc koteswarao

Name	Null?	Type
------	-------	------

BEST_FRIENDS_NAME		VARCHAR2(15)
-------------------	--	--------------

PHONENO		NUMBER(10)
---------	--	------------

ADDRESS		VARCHAR2(20)
---------	--	--------------

SALARY		NUMBER(6)
--------	--	-----------

SQL> insert into koteswarao values('manikanta',8688598110,'vinukonda',180000);

1 row created.

SQL> insert into koteswarao values('anil_kumar',9440774380,'vinukonda',140000);

1 row created.

SQL> insert into koteswarao values('chandrajith',7358959739,'namakal',900000);

1 row created.


```
SQL> insert into koteswarao values('shankar-bhai',8074862610,'kadapa',600000);
```

1 row created.

```
SQL> insert into koteswarao values('nithish_kumar',9347734009,'guntur',800000);
```

1 row created.

```
SQL> select * from koteswarao;
```

BEST_FRIENDS_NAME	PHONENO	ADDRESS	SALARY
-------------------	---------	---------	--------

manikanta	8688598110	vinukonda	180000
-----------	------------	-----------	--------

anil_kumar	9440774380	vinukonda	140000
------------	------------	-----------	--------

chandrajith	7358959739	namakal	900000
-------------	------------	---------	--------

shankar-bhai	8074862610	kadapa	600000
--------------	------------	--------	--------

nithish_kumar	9347734009	guntur	800000
---------------	------------	--------	--------

```
SQL> update koteswarao set  
salary=190000 where best_friends_name='manikanta';
```

1 row updated.

```
SQL> select * from koteswarao;
```

BEST_FRIENDS_NAME	PHONENO	ADDRESS	SALARY
-------------------	---------	---------	--------

manikanta	8688598110 vinukonda	190000
anil_kumar	9440774380 vinukonda	140000
chandrajith	7358959739 namakal	900000
shankar-bhai	8074862610 kadapa	600000
nithish_kumar	9347734009 guntur	800000

SQL> delete from koteswarao where best_friends_name='anil_kumar';

1 row deleted.

SQL> select * from koteswarao;

BEST_FRIENDS_NA	PHONENO ADDRESS	SALARY
manikanta	8688598110 vinukonda	190000
chandrajith	7358959739 namakal	900000
shankar-bhai	8074862610 kadapa	600000
nithish_kumar	9347734009 guntur	800000

SQL> select best_friends_name from koteswarao;

BEST_FRIENDS_NA

manikanta
chandrajith
shankar-bhai

nithish_kumar

```
SQL> select distnict address from koteswarao;
```

```
select distnict address from koteswarao
```

*

ERROR at line 1:

ORA-00904: "DISTNICT": invalid identifier

```
SQL> select distinct address from koteswarao;
```

ADDRESS

vinukonda

guntur

kadapa

namakal

```
SQL> select * from koteswarao where salary between 150000 and 999999;
```

BEST_FRIENDS_NA	PHONENO	ADDRESS	SALARY
-----------------	---------	---------	--------

manikanta	8688598110	vinukonda	190000
-----------	------------	-----------	--------

chandrajith	7358959739	namakal	900000
-------------	------------	---------	--------

shankar-bhai	8074862610	kadapa	600000
--------------	------------	--------	--------

nithish_kumar	9347734009	guntur	800000
---------------	------------	--------	--------


```
SQL> select address as native_place from koteswarao;
```

```
NATIVE_PLACE
```

```
-----
```

```
vinukonda
```

```
namakal
```

```
kadapa
```

```
guntur
```

```
SQL> select phoneno from koteswarao order by salary asc;
```

```
PHONENO
```

```
-----
```

```
8688598110
```

```
8074862610
```

```
9347734009
```

```
7358959739
```

```
SQL> select salary from koteswarao where best-friends_name like '_0%';
```

```
select salary from koteswarao where best-friends_name like '_0%'
```

```
*
```

```
ERROR at line 1:
```

```
ORA-00904: "FRIENDS_NAME": invalid identifier
```


SQL> select salary from koteswarao where best_friends_name like '_0%';

no rows selected

SQL> select best_friends_name from koteswarao where salary=190000 and
phoneno=8688598110;

BEST_FRIENDS_NAME

manikanta

SQL> create user koteswarao identified by kunal;

User created.

SQL> grant all privileges to koteswarao;

Grant succeeded.

SQL> revoke all privileges from koteswarao;

Revoke succeeded.

SQL> commit;

Commit complete.